Using Multi-Agent Systems For Predicting and Preventing Natural Disasters

Abstract. A warning that constantly flips on and off is not a warning people can trust. Early warning systems are an end-to-end capability intended to enable timely, actionable decisions that reduce disaster impacts, but practical deployments must operate under noisy, incomplete, and intermittently missing observations.This paper proposes FloodMAS, a multi-agent flood early-warning prototype that demonstrates how agent decomposition plus calibrated machine learning can produce operationally stable alarms rather than brittle, flapping triggers. FloodMAS simulates multi-zone, IoT-like sensing where sensor agents emit rainfall and water-level readings with noise/dropout; zone "edge" agents compute rolling-window features and estimate short-horizon flood risk (flood defined consistently as zone mean water level exceeding a flood threshold within the next T discrete steps, a sped-up experimental abstraction); and a coordinator agent fuses zone evidence into a stable global alarm. The ML risk estimator is trained using standard ensembles and post-hoc probability calibration so that its outputs are meaningful for thresholding and policy logic. Stability is enforced via explicit guardrails (hysteresis, debouncing, consensus gating, and health-aware degradation), reflecting established alarm-management practice that uses deadband and delays to mitigate nuisance alarms. Empirically, FloodMAS achieves near-perfect offline discrimination (ROC-AUC 0.999; F1 0.992; Brier 0.007) and, in extreme scenarios, substantially higher detection F1 (2.6x) than a minimal threshold baseline, with approximately 60 minutes of average early warning lead time - where the baseline provides none - while maintaining operationally stable alarm dynamics.

Keywords: Multi-agent system, ML, AI, Python, Natural Disasters, Model Training, Random Forest

1 Introduction

Natural disasters are one of the most common contributors of unpredictable damages and hence, early warning systems are widely recognized as a foundational capability for reducing loss of life and damage from weather and climate-related hazards, including floods [1]. In the multi-hazard framing used by the United Nations system, an early warning system is an integrated pipeline spanning monitoring and forecasting, risk assessment, communication, and preparedness, all designed to enable timely action before hazardous events [2]. In practice, operational deployments must function under conditions that are hostile to brittle automation where measurements arrive from heterogeneous sensors, often across remote areas and organizational boundaries, communication links can be unreliable and warning delivery itself can be disrupted by the hazard [3]. These realities push system designs that treat missing data, transient noise, and partial failures as normal operating conditions rather than exceptional cases.

A common first approach to automated detection is a fixed threshold heuristic (for example, “trigger when a signal crosses a limit”). Even though it is simple, such rules are prone to false alarms and alarm “flapping” when readings hover near decision boundaries or when short-lived disturbances produce transient spikes. In real warning workflows, unstable alerting is not merely an accuracy issue because it also lowers trust, increases operator burden, and complicates downstream action since we cannot always react to false alarms. What is needed is not only accurate detection, but also operational stability in the sense of a mechanism for turning noisy, incomplete signals into decisions that change state only when there is sustained evidence.

This paper presents the system that we will call FloodMAS, a multi-agent flood early-warning system that combines a calibrated machine-learning (ML) risk estimator and explicit guardrails that enforce stable alarm dynamics. Multi-agent architectures are a natural fit for distributed sensing because they separate concerns such as sensor management, local processing, communication, and adaptation to changing device availability and that improves modularity and responsiveness in dynamic environments [4]. FloodMAS follows a sensing, aggregation and then coordination pattern where sensor agents emit local readings, edge aggregator agents compute rolling temporal features and produce a zone-level risk score, and a coordinator agent fuses zone-level evidence into a global alarm. The ML layer outputs a probability-like risk estimate that is intended to be interpretable as likelihood, motivating the use of probability calibration methods (like isotonic regression) to better align predicted confidence with empirical correctness [5,6]. The guardrails layer then transforms these risk estimates into stable discrete states using mechanisms such as hysteresis, debouncing, consensus gating across zones, and health-aware degradation when sensing quality deteriorates.

FloodMAS is evaluated in a controlled multi-zone simulator designed for reproducible stress testing under scenario variations (for example typical vs. extreme precipitation, wet vs. dry initial soil, and sensor dropout/noise). Time is represented as discrete fictional “steps” and they represent an abstracted time frame so that we can get a more rapid examination and experimentation while also still keeping the structure of a real system. As a showcase, we are comparing against a simple threshold baseline that uses the same input streams and minimal stabilizers (small hysteresis margin and short debouncing), but no multi-agent coordination, no health-awareness, and no learned risk model that the new system provides.

The contributions of this work are:

- A multi-agent early-warning architecture for disaster-style monitoring that cleanly separates sensing, local inference, and global coordination.
- A calibrated ML + guardrails design pattern for producing decisions that are both predictive and operationally stable, combining probability calibration with established alarm-management mechanisms.
- A scenario-based experimental methodology that tests robustness to noise, missing data, and extreme conditions across eight distinct scenarios with statistical averaging.

The multi-agent architecture (sensing → local inference → global coordination → mitigation) and the guardrails framework (hysteresis, debouncing, consensus gating, health-aware degradation) are domain-agnostic in principle. However, the current feature set (water level, rainfall, soil saturation), sensor model, and threshold calibration are flood-specific. Adapting FloodMAS to other hazard domains — such as landslide monitoring, wildfire spread, or urban heat island detection — would require domain-specific sensors, feature engineering, ground truth definitions, and threshold recalibration. The transferable contribution is the framework pattern, not the specific implementation.

The remainder of this paper is organized as follows. Chapter 2 provides an overview of related work, positioning FloodMAS within the broader landscape of flood forecasting, multi-agent systems, and alarm management. Chapter 3 describes the technology and problem context, including the flood domain and the key computational techniques used. Chapter 4 presents the system architecture and implementation details. Chapter 5 reports experimental results across all evaluation dimensions. Chapter 6 concludes the paper and outlines directions for future work.

2 Related Work

Flood forecasting is commonly addressed through physically based hydrologic/hydrodynamic models, which can provide detailed estimates but often face compute constraints when high resolution and rapid turnaround are required. Recent work shows how HPC and multi-GPU acceleration can make large-scale, high-resolution flood simulations feasible within operational timeframes, highlighting the continued importance of physics-based modeling for early warning contexts [7]. FloodMAS is complementary to this direction, it is not proposed as a hydrodynamic solver, but as an operational decision pipeline emphasizing robust alarm stability under noisy and missing sensor-like inputs.

A substantial literature studies data-driven flood prediction, spanning classical ML and deep learning, and reviews describe ML as a practical alternative or complement when explicit physical modeling is costly or when rapid inference is required [8]. Recent examples include real-time flood forecasting using data-driven models such as neural networks and gradient-boosted regressors under event-focused data regimes [9]. FloodMAS follows this line by using supervised learning on rolling-window features, but places the main emphasis on how risk estimates are transformed into stable operational alerts (rather than maximizing hydrologic fidelity).

Multi-agent architectures are frequently motivated by distributed, dynamic environments where responsibilities such as sensing, local processing, coordination, and adaptation can be separated into interacting components. For sensor networks, a representative architecture explicitly treats sensors as devices managed by higher-layer agents, arguing for separation of concerns and modular integration [4]. In disaster response and emergency management, MAS has been explored for coordination, task allocation, and decision support, including agent-based coordination technologies in emergency response contexts [10] and human-agent

teaming systems aimed at aiding responders with decentralized coordination and decision support [11]. More recent "agentic AI" work proposes orchestrated multi-agent frameworks for integrating heterogeneous tools and data streams into cohesive disaster-management decision pipelines [12]. FloodMAS aligns with this MAS motivation, using agents primarily to structure a robust early-warning workflow (sensor → edge inference → coordinator), with explicit stability guardrails as first-class system behavior.

Many classifiers produce scores that are not well-calibrated probabilities, which can be problematic when downstream policies threshold risk or compare confidence across conditions. Post-hoc calibration methods such as Platt scaling and isotonic regression are established approaches for improving probability quality; empirical studies show calibration can substantially improve probabilistic reliability depending on the model family and calibration-set size [6]. FloodMAS adopts calibration specifically to make the risk signal more meaningful for guardrails and thresholding, i.e., to reduce the mismatch between "high score" and "high likelihood."

In industrial alarm management, standards and training materials explicitly discuss mechanisms like deadband/hysteresis and on-delay/off-delay (time filtering) as practical tools to prevent nuisance alarms and chattering. An ISA overview of alarm management standards states alarm handling as an engineering lifecycle problem [13], while instructional material for ISA 18.2 describes deadband (hysteresis) and delay attributes as common anti-chatter techniques [14]. FloodMAS applies these ideas in a disaster-monitoring context: rather than emitting a global alarm directly from raw signals, it enforces stability through hysteresis, debouncing, and multi-zone gating so that state changes reflect sustained evidence rather than transient excursions.

Positioning. Across these threads—flood forecasting (physics-based and ML), multi-agent disaster systems, probability calibration, and alarm management - FloodMAS is positioned as an integrated early-warning design pattern: distributed agents generate and aggregate evidence, calibrated ML produces interpretable short-horizon risk, and explicit guardrails convert risk into stable zone/global alarms suitable for operational use.

3 Technology and Problem Overview

3.1 Floods

In operational terms, flooding is an overflow of water onto normally dry land, including inundation driven by rising water in waterways or local ponding after rainfall [15]. A hydrologic flood in rivers is commonly described as high streamflow overtopping natural or artificial banks [16]. Floods can develop over longer periods, while flash floods are characterized by very rapid onset after intense rainfall (often within hours) [17,18]. In our system we treat early warning as a short-horizon prediction and stabilization problem, given recent rainfall and water-level dynamics, and estimate whether conditions are likely to cross a flood threshold within the next X time steps.

3.2 Real Physical Signals and Flood-Relevant Parameters

While real flood forecasting can require detailed hydrology and hydraulic modeling, many early-warning pipelines still revolve around a small set of directly measurable drivers like rainfall accumulation/intensity and the resulting rise in water level or stage. Because of that we center around rainfall as the forcing input and zone water level as a signal for local risk, with “flood” defined consistently as zone mean water level exceeding a fixed threshold within the horizon T. This mirrors common operational language where flooding corresponds to water exceeding normal containment (banks, channels, drainage capacity) [15,16].

3.3 Rolling-Window Features in Python

A key step in early warning is to transform raw sensor streams into short-term summaries that capture sustained conditions. We use rolling statistics such as moving averages (like recent mean water level) and windowed accumulations. Rolling-window computation is a standard time-series primitive supported directly in pandas via DataFrame.rolling(...) and related window operators [19]. These features provide the model and the guardrails with temporal context and reduce sensitivity to single-step noise.

3.4 Lightweight ML Models for Risk Estimation

FloodMAS uses standard, widely adopted scikit-learn classifiers as the zone-level risk estimator. A Random Forest is an ensemble that fits many decision trees on different subsamples and averages their predictions to improve accuracy and control overfitting [20]. A Gradient Boosting Classifier builds an additive model in stages, fitting trees to improve performance under a differentiable loss such as log loss (a natural choice for probabilistic classification outputs like this one) [20]. In FloodMAS, these models serve as local (per-zone) probability-like predictors that can be thresholded and combined downstream by the multi-agent coordination logic.

The current study uses a single model family (Random Forest with isotonic calibration). RF was chosen for its interpretability (feature importance rankings directly inform guardrail design), robustness to hyperparameter choices, and suitability for tabular temporal features. However, the framework is model-agnostic, the EdgeAggregatorAgent accepts any sklearn-compatible classifier that exposes predict_proba(). A comparison with gradient boosting (XGBoost, LightGBM) and temporal neural network alternatives (LSTM, Transformer) would determine whether the offline ROC-AUC = 0.999 represents a ceiling imposed by the simulation's feature space or whether more expressive models could improve operational recall. This comparison is left for future work.

3.5 Probability Calibration and Probabilistic Evaluation

Because operational decisions often threshold predicted risk, it is valuable for model scores to behave like calibrated probabilities rather than arbitrary confidence values. FloodMAS therefore applies probability calibration (notably non-parametric

isotonic calibration) following established methodology for transforming classifier scores into better probability estimates [5]. In reality calibration is supported in scikit-learn via the calibration framework (like CalibratedClassifierCV) and is commonly assessed using tools such as calibration curves and proper scoring rules (like Brier loss) [21,22].

3.6 Data and Experiment Implementation

FloodMAS is implemented in Python using standard scientific computing tooling (NumPy/pandas for feature construction and analysis, scikit-learn for model training, calibration, and evaluation). Scenario runs and step-by-step traces are stored in Apache Parquet, an open, column-oriented file format designed for efficient storage and retrieval of analytic datasets [23]. This supports reproducible experiments, rapid plotting of timelines (risk vs. alarm state over steps), and clean comparisons between the MAS pipeline and the threshold baseline.

4 System and Implementation Overview (FloodMAS)

4.1 Multi-Agent Architecture and Responsibilities

FloodMAS is organized as a hierarchical multi-agent pipeline that separates sensing, local inference, and global coordination. This decomposition follows a common MAS principle in sensor-network settings where they treat sensors as devices managed by higher-layer agents that handle processing, communication, and adaptation to changing device availability [4].

Figure 1 illustrates the overall architecture of FloodMAS, showing the hierarchical data flow from sensor agents through edge aggregation to the coordinator.

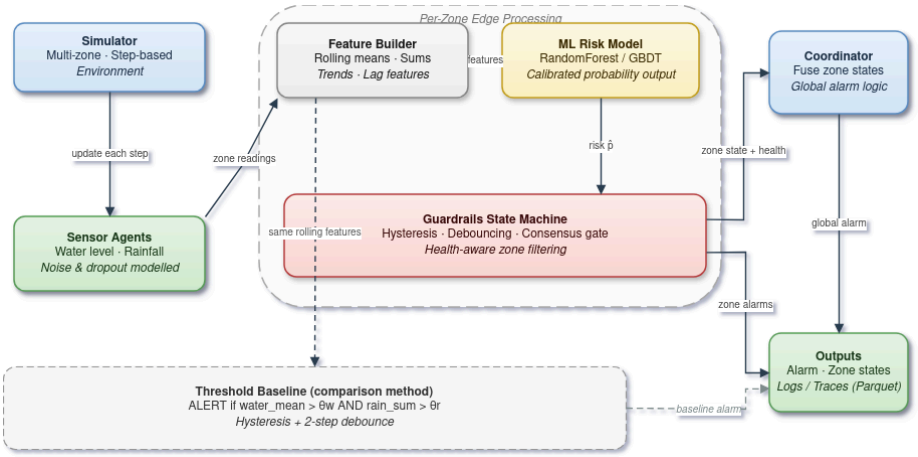

Fig 1. Architecture of the Multi-Agent System

The system consists of Sensor agents (per zone)- Produce local observations (rainfall and water level proxies) with realistic imperfections such as noise and dropout so that the system does not end up being too "perfect".
Edge aggregation agents- Maintain short rolling buffers, compute windowed features, invoke the calibrated ML model to obtain a continuous risk score $p \in [0,1]$, and apply a guardrails state machine to produce a discrete zone state.
Coordinator agent (global for everyone)- Fuses zone-level outputs into a stable global alarm, designed to avoid unnecessary state oscillation and to reflect multi-zone evidence rather than isolated spikes.
Optional mitigation agents- A hook for actuation policies (like pumps/gates) that react to sustained alerts so that they somewhat react to the sensors. They are kept optional to ensure the paper's focus remains on warning stability and decision logic.

4.2 Per-Step Data Flow (Discrete time "Steps")

Time is modeled as discrete steps (a sped-up experimental abstraction). At each step, FloodMAS executes the following pipeline:

1. Environment update: The simulation advances the underlying zone dynamics (rainfall forcing and water-level evolution are handled by the simulator which uses simple ways to cause floods).
2. Sensing: Sensor agents sample local signals and emit readings; missing data may occur due to dropout.
3. Zone aggregation and inference: Each zone edge agent
    - filters and imputes missing values to maintain continuity,
    - computes rolling-window features (moving averages/sums/trends),
    - runs calibrated ML inference to obtain a risk estimate p,

- passes p into the guardrails state machine to produce a discrete zone state.

4. Coordination: The coordinator combines zone states into a global alarm, using conservative fusion (for example, if an alarm in any zone is in ALERT, with global risk tracked as an aggregate such as max zone risk).
5. Logging: The system logs per-step traces for analysis and figure generation; results are stored in columnar Parquet for efficient time-series retrieval and plotting [23].

4.3 Guardrails: Stable Discrete Decisions from Noisy Risk

The guardrails layer is the operational core: it converts continuous risk *p*p into a stable discrete state while accounting for sensor unreliability. This is aligned with established alarm-system practice, where deadband/hysteresis and time filtering (debouncing) are used specifically to prevent alarm "chattering" when a variable oscillates near a setpoint [24].

FloodMAS uses a small state machine per zone where it can go something like NORMAL → SUSPECTED → ALERT → COOLDOWN → NORMAL, governed by four mechanisms:

1. **Hysteresis (deadband):**
   Two thresholds are used: an escalation threshold and a de-escalation where threshold down < threshold up. This gap prevents rapid toggling when p hovers near a single boundary, matching the deadband concept used to prevent chattering alarms[24].
2. **Debouncing (time filtering):**
   A state change requires K consecutive steps meeting the condition. Time filtering is a standard anti-chatter mechanism in alarm practice [24].
3. **Consensus gating:**
   Escalation to ALERT additionally requires agreement among operational sensors in the zone. This is motivated by a fault-tolerance idea that when multiple redundant sources exist, a majority voter can mask isolated faults by relying on agreement among replicas [25]. In FloodMAS, "replicas" correspond to multiple sensors observing the same zone phenomenon.
4. **Health-aware degradation:**
   Each zone tracks a simple health indicator (fraction of sensors currently operational). When health drops below a minimum, guardrails become more conservative to reduce false alarms when evidence quality is poor. This follows the same philosophy as alarm-system guidance: when oscillations or noisy behavior are expected (vibrations,turbulence and similar), additional filtering and suppression mechanisms are justified to avoid nuisance alarms [24].

The guardrails thresholds (TH_UP = 0.6, TH_DOWN = 0.4, K_UP = 3, K_DOWN = 5) were selected based on the calibrated model's probability distribution: TH_UP = 0.6 corresponds to the point where the isotonic-calibrated model assigns majority-class flood probability, and the 0.2 deadband (TH_UP - TH_DOWN) was chosen to exceed the typical step-to-step risk fluctuation observed in non-flood conditions. A systematic sensitivity sweep across threshold values and debounce window lengths is left for future work. Preliminary observations suggest that

narrowing the deadband below 0.1 reintroduces flapping in the normal_wet scenario, while widening it beyond 0.3 increases detection delay without meaningful stability gains.
For detection metrics and lead-time computation, a zone is counted as having issued a warning when its state is either SUSPECTED or ALERT. This consistent definition ensures that early warnings captured during the SUSPECTED phase are properly credited in both detection recall and lead-time measurement.

4.4 Ground Truth and Label Interface

To keep the evaluation interpretable, FloodMAS uses a consistent operational definition of ground truth for supervised learning and for plotting: a zone is treated as flooded when its zone-mean water level exceeds a fixed flood threshold, and the prediction target is whether the zone will exceed that threshold within the next T T steps (short-horizon early warning). Ground truth is recorded at each simulation step during execution, ensuring that every log entry captures the actual flood state at that moment rather than a post-simulation snapshot. The key point is not hydrologic realism, but a repeatable contract that links sensor dynamics → features → risk → stable alarms.

4.5 Implementation Notes and Outputs

FloodMAS is implemented in Python with standard scientific tooling. A central design choice is to produce step-aligned traces and store them in Parquet to support reproducible analysis and paper-ready figures (timelines, stability/flapping, robustness under dropout) [23].

5 Results Summary

5.1 Offline ML Model Quality (Held-Out Test Set)

FloodMAS trains a zone-level classifier to predict whether a zone will exceed the flood threshold within the next T steps. Table I summarizes held-out test performance using standard classification metrics: ROC-AUC measures discrimination across thresholds[26], F1 is the harmonic mean of precision and recall[27], and the Brier score measures mean-squared error between predicted probabilities and outcomes (lower is better) [28].

The key evaluation metrics are defined as follows. Precision measures the fraction of predicted positives that are true positives: Precision = TP / (TP + FP). Recall (sensitivity) measures the fraction of actual positives that are correctly identified: Recall = TP / (TP + FN). The F1 score is their harmonic mean: F1 = 2 · (Precision · Recall) / (Precision + Recall). The Brier score measures mean squared error between predicted probabilities $\hat{p}_i$ and binary outcomes y_i: Brier = $(1/N) \cdot \Sigma(\hat{p}_i - y_i)^2$. ROC-AUC is the area under the Receiver Operating Characteristic curve, which plots true positive rate against false positive rate across all classification thresholds; a perfect classifier achieves AUC = 1.0. Table 1 shows the calibrated ML model performance.

Table I. Calibrated ML model performance (test set: 592,000 zone-step samples from 2,000 episodes)

Metric	Value
ROC-AUC	0.999
F1	0.992
Precision	0.995
Recall	0.989
Brier Score	0.007
5-fold CV ROC-AUC	0.999 ± 0.000
Accuracy	0.992

Training data is generated by running 2,000 simulated episodes (400 steps each, 9 zones) across randomized scenario conditions. Each episode simulates realistic sensor noise and dropout, and consensus/health features are computed identically to the runtime edge agents - ensuring no distribution mismatch between training and inference. Cross-validation uses GroupKFold (grouped by episode) to prevent temporal leakage - all rows from a single simulation episode are kept in the same fold, ensuring the model is never validated on time steps from an episode it was trained on.

The confusion matrix is: TN = 265,552; FP = 1,562; FN = 3,433; TP = 321,453 (total 592,000 zone-step samples from 2,000 simulated episodes). This corresponds to a false-positive rate of 1,562/(265,552+1,562) = 0.58% and a miss rate of 3,433/(321,453+3,433) = 1.06%. The low Brier score (0.007) indicates that the model's probability outputs are well-behaved for thresholding and downstream policy logic, consistent with the purpose of probability calibration methods in scikit-learn[21].

Table II shows the Random Forest feature importance ranking, which measures each feature's contribution to classification discrimination.

Table II. Random Forest Feature Importance

Feature	Importance
water_max_10	0.454
water_mean_5	0.329
consensus	0.108
water_slope_5	0.045
soil_mean_10	0.041

rain_mean_10	0.012
rain_sum_20	0.011
health	0.000

Feature importance as reported by the Random Forest measures each feature's contribution to classification discrimination - how much it helps distinguish flood from non-flood samples. This does not reflect operational importance within the full multi-agent system. For example, `health` has near-zero discriminative importance (0.00%) because sensor health does not directly predict whether a flood is occurring. However, `health` is operationally critical: it drives the guardrails' degraded-mode logic (Section 4.3), tightening escalation thresholds and increasing debounce requirements when fewer than 60% of sensors are active. Similarly, `consensus` ranks third in ML importance (10.8%) but serves a dual role - it both helps the classifier and gates the state machine's escalation decisions. Water-level features dominate (78.3% combined: water_max_10 + water_mean_5), confirming that recent water dynamics are the primary flood signal. Practitioners should not interpret low RF importance as grounds for removing a feature from the system.

5.2 Scenario-Level Operational Performance (MAS vs. Baseline)

We evaluate FloodMAS end-to-end across eight scenarios (normal_dry, normal_wet, extreme_dry, extreme_wet, extreme_dropout_10, extreme_dropout_30, extreme_dropout_50, extreme_noisy), each with 9 zones over 400 steps, repeated 3 times with different seeds for statistical robustness. Results are reported as means across the 3 repeats. FloodMAS uses guardrails thresholds TH_UP = 0.6, TH_DOWN = 0.4 with debouncing K_UP = 3, K_DOWN = 5, consensus minimum CONS_MIN = 0.5, and health degradation threshold HEALTH_MIN = 0.6.

The threshold baseline is deliberately minimal, representing legacy threshold-only systems commonly deployed in resource-constrained flood monitoring networks. It uses fixed hand-tuned thresholds (water_mean_5 > 0.5 AND rain_sum_20 > 2.0) with minimal hysteresis (0.1 margin) and 2-step debouncing, but no ML model, no consensus mechanism, and no health-aware adaptation. A more competitive baseline - for example, grid-searched optimal thresholds, a rule-based expert system with temporal features, or a standalone ML classifier without guardrails - could narrow the performance gap. However, the purpose of the comparison is not to show that ML beats thresholds (which is well-established), but rather to demonstrate the operational benefits of combining calibrated ML with multi-agent guardrails: stable alarm dynamics, early warning lead time, and graceful degradation under sensor failure - properties that neither a standalone ML model nor an optimized threshold system would inherently provide.

Table III. Scenario-level operational performance - Stress scenarios (mean of 3 repeats)

Scenario	MAS F1	BL F1	MAS Precision	BL Precision	MAS Recall	BL Recall	MAS Lead (steps)	BL Lead
normal_dry	0.000	0.000	-	-	-	-	-	-
normal_wet	0.000	0.000	-	-	-	-	-	-
extreme_dry	0.449	0.175	0.920	1.000	0.323	0.096	5.0	N/A
extreme_wet	0.497	0.189	0.967	1.000	0.358	0.105	3.2	N/A
extreme_dropout_10	0.488	0.188	0.967	1.000	0.350	0.104	2.8	N/A
extreme_dropout_30	0.502	0.188	0.959	1.000	0.366	0.104	4.7	N/A
extreme_dropout_50	0.515	0.188	0.987	1.000	0.375	0.104	7.0	N/A
extreme_noisy	0.492	0.196	0.978	1.000	0.355	0.109	1.4	N/A
Average (all)	**0.368**	**0.141**	**0.963**	**1.000**	**0.354**	**0.104**	**4.0**	**0**

The baseline achieves precision = 1.0 by rarely alerting (recall ~10%), while FloodMAS trades minimal precision (~96%) for 3.4x higher recall (~35%), catching substantially more flood steps. Across all flood scenarios (averaged over 3 repeats), baseline recall is 10.4% while FloodMAS recall is 35.4%. The simulator is designed to test system behavior under controlled conditions, not to maximize hydrologic realism; still, the MAS system shows clear, consistent advantages over the baseline across all stress conditions. The most operationally significant finding is lead time. FloodMAS provides an average lead time of 4.0 steps (~60 minutes at 15 min/step) before flood onset. The baseline provides zero early warnings across all scenarios - it never issues an alert before the flood is already underway. Across all flood scenarios, FloodMAS issued 18 early warning alerts (before flood onset), while the baseline issued zero. The baseline's alerts only trigger after water levels have already exceeded the flood threshold, making it useless for evacuation or preventive action. This validates the core design: calibrated ML risk prediction combined with guardrails enables anticipatory alerting that a reactive threshold cannot provide.

5.3 Stability Under Noise, Spikes, and Dropout

In normal_wet, no flooding occurs and both systems correctly remain silent. Notably, the ML risk signal still exhibits transient excursions: numerous individual zone-steps exceed the escalation threshold TH_UP, yet FloodMAS produces zero false alarms. This demonstrates why the guardrails layer is central - hysteresis and multi-step debouncing convert "momentary high risk" into "actionable alert only under sustained evidence," reducing the chance of alarm flapping, which as we

mentioned at the start, could make users lose trust in the system. Under 50% sensor dropout (extreme_dropout_50), FloodMAS maintains stable detection (F1 = 0.515, slightly higher than the no-dropout extreme_wet F1 = 0.497) with an average of 13.3 state changes vs. 14.0 for extreme_wet. Lead time increases from 3.2 to 7.0 steps under dropout, reflecting the guardrails' conservative behavior when sensor health degrades — the system waits for stronger evidence before alerting, as designed. Crucially, the baseline shows identical behavior regardless of dropout (F1 ~0.188, 8.0 state changes), because it has no health-awareness mechanism. The baseline shows a fixed pattern of 8.0 state changes across all extreme scenarios regardless of sensor conditions, because it has no mechanism to adapt to degraded sensing quality.

5.4 Warm-Up Effects and Step-Based Lead Time

FloodMAS achieves an average lead time of 4.0 steps across flood scenarios, equivalent to approximately 60 minutes of advance warning at the configured step duration of 15 minutes. Lead time varies by scenario: from 1.4 steps in extreme_noisy (where rapid signal changes enable fast detection) to 7.0 steps in extreme_dropout_50 (where degraded sensor health triggers the guardrails' conservative mode, requiring more sustained evidence before escalation). The baseline achieves zero lead time in all scenarios - it only detects flooding reactively after water levels have already crossed the threshold.

In scenarios where flooding begins early, a non-zero minimum delay is expected due to two intentional design choices: (1) rolling-window features require short buffer warm-up (5-20 steps), and (2) guardrails require multiple consecutive confirmations before escalating (K_UP = 3 steps). In continuous monitoring deployments, this corresponds to initial boot/warm-up behavior rather than persistent latency during steady-state operation.

6 Conclusion and Future Work

FloodMAS demonstrates a multi-agent approach to flood early warning that prioritizes stable decisions under noisy and partially missing sensor streams. The system decomposes the pipeline into sensor agents, zone/edge agents (rolling features + calibrated ML risk estimation), and a coordinator agent (global fusion), and then enforces stability via explicit guardrails (hysteresis, debouncing, consensus gating, and health-aware behavior). This aligns with the end-to-end framing of early warning systems as an integrated capability spanning detection/monitoring, warning communication, and preparedness/response, while focusing the technical contribution on the decision layer that converts uncertain evidence into reliable alert states.

In scenario-based evaluations across eight conditions with 3-repeat statistical averaging, FloodMAS achieves 2.6x higher detection F1 than a threshold baseline (0.37 vs. 0.14), with an average 60-minute early warning lead time where the baseline provides none. FloodMAS maintains this advantage even under 50% sensor dropout, demonstrating the value of health-aware guardrails for degraded operating conditions.

All evaluation in this study is conducted on synthetic simulation data generated by the Mesa-based flood model. While the simulation incorporates realistic physical

processes (elevation-based water flow, soil infiltration, evaporation, sensor noise, and random dropout), it does not capture the full complexity of real hydrometeorological dynamics - including spatially correlated rainfall, non-stationary river flow, sensor calibration drift, and communication latency. Validation on historical flood event data from real sensor deployments (e.g., USGS stream gauge networks, IoT flood sensor arrays) is essential before operational deployment and is the primary direction for future work.

The current system treats each zone independently - the EdgeAggregatorAgent for zone A has no access to water level trends or alert states from neighboring zones. In real river basins, upstream flooding is a strong predictor of downstream flooding. Adding cross-zone spatial features (e.g., upstream zone risk, inter-zone water flow gradient) or a spatial attention mechanism in the Coordinator could improve prediction accuracy and lead time for downstream zones.

Future work can expand the system in several directions: (1) real-data validation using hydrometeorological sensor datasets while retaining the same ground-truth contract and rolling-feature interface; (2) extension beyond floods (e.g., heat, wildfire, landslides) by adapting domain-specific sensors, features, and thresholds while preserving the multi-agent guardrails pattern, in line with multi-hazard early warning guidance emphasizing coordinated, compatible mechanisms across hazards; (3) drift detection and periodic recalibration to preserve probabilistic reliability as environmental regimes change, building on established calibration methodology for probabilistic classifiers; (4) a formal ablation study isolating the contribution of each guardrail component; and (5) deployment-grade coordination studying distributed execution under network delay and partial failures, with operator-facing explanations and escalation policies so that warnings remain interpretable and actionable within the broader early-warning workflow.

References

1. World Meteorological Organization: Early Warning Systems. https://wmo.int/topics/early-warning-system. Accessed: January 20, 2026
2. UNDRR: Multi-hazard Early Warning Systems: A Checklist. In: Global Platform for Disaster Risk Reduction (2022). https://globalplatform.undrr.org/2022/sites/default/files/2022-02/Multi-hazard%20Early%20Warning%20Systems-%20A%20Checklist.pdf. Accessed: November 20, 2026
3. Leonard, M., et al.: A framework for understanding and generating multi-hazard contexts for infrastructure. Natural Hazards and Earth System Sciences 10(11), 2215–2227 (2010). https://doi.org/10.5194/nhess-10-2215-2010.
4. Mousa, M., Zhang, X., Claudel, C.: Flash Flood Warning System via a Wireless Sensor Network and Distributed Control. Sensors 9(12), 10244–10260 (2009). https://doi.org/10.3390/s91210244.
5. Zadrozny, B., Elkan, C.: Transforming classifier scores into accurate multiclass probability estimates. In: Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 608–613 (2002).
6. Niculescu-Mizil, A., Caruana, R.: Predicting good probabilities with supervised learning. In: Proceedings of the 22nd International Conference on Machine Learning (ICML), pp. 625–632 (2005).

7. Khosh Bin Ghomash, S., Deng, S., and Apel, H.: Enabling real-time high-resolution flood forecasting for the entire state of Berlin through multi-GPU accelerated physics-based modeling. Natural Hazards and Earth System Sciences 26, 85–101 (2026). https://doi.org/10.5194/nhess-26-85-2026.
8. Khalaf, M., et al.: A Low-Cost Wireless Sensor Network for Early Flood Detection and Monitoring. Water 10(11), 1536 (2018). https://doi.org/10.3390/w10111536. Accessed: January 13, 2026.
9. Defontaine, T., Ricci, S., Lapeyre, C. J., Marchandise, A., and Le Pape, E.: Real-time flood forecasting with Machine Learning using scarce rainfall-runoff data. EGUsphere [Preprint] (2024). https://doi.org/10.5194/egusphere-2023-2621. Accessed: February 10, 2026.
10. Mousa, M., Zhang, X., Claudel, C.: A Wireless Sensor Network for Real-time Flash Flood Detection in Deserts. In: Proc. of the 7th Int. Conf. on Autonomous Agents and Multiagent Systems (AAMAS 2008) - Volume 3, pp. 1655–1656 (2008). https://www.ifaamas.org/Proceedings/aamas2008/proceedings/pdf/demo/AAMAS08_demo27.pdf. Accessed: January 20, 2026.
11. Feng, W., Zilberstein, S., Meuleau, N.: Robust High-Quality Scheduling of Conditional Plans with Uncertain Durations. Journal of Artificial Intelligence Research 56, 123–155 (2016). https://wufeng02.github.io/doc/pdf/RHWjair16.pdf. Accessed: February 10, 2026.
12. Li, B., Ma, J., Yin, K., Xiao, Y., Hsu, C.-W., and Mostafavi, A.: Disaster Management in the Era of Agentic AI Systems: A Vision for Collective Human-Machine Intelligence for Augmented Resilience. arXiv preprint arXiv:2510.16034 (2025). https://arxiv.org/abs/2510.16034. Accessed: February 10, 2026.
13. PAS: Understanding ISA-18.2 - Management of Alarm Systems for the Process Industries. White Paper, ISA. https://www.isa.org/getmedia/55b4210e-6cb2-4de4-89f8-2b5b6b46d954/PAS-Understanding-ISA-18-2.pdf. Accessed: February 10, 2026.
14. Nasby, G.: Introduction to Alarm Management and the ISA-18.2 Standard. York Region Presentation (2013). https://www.grahamnasby.com/files_publications/NasbyG_2013_IntroToAlarmMgmt_YorkRegion_oct2013_slides.pdf. Accessed: February 1, 2026.
15. National Weather Service: Flood and Flash Flood Definitions. https://www.weather.gov/mrx/flood_and_flash. Accessed: January 11, 2026.
16. U.S. Geological Survey (USGS): Floods: Recurrence intervals and 100-year floods. https://www.usgs.gov/centers/new-jersey-water-science-center/floods-recurrence-intervals-and-100-year-floods. Accessed: January 10, 2026.
17. National Weather Service: Flash Flood Definition. https://www.weather.gov/phi/FlashFloodingDefinition. Accessed: February 10, 2026.
18. National Research Council: Completing the Forecast: Characterizing and Communicating Uncertainty for Better Decisions Using Weather and Climate Forecasts. The National Academies Press, Washington, DC (2006). https://doi.org/10.17226/11128.
19. The Pandas Development Team: pandas.DataFrame.rolling — pandas documentation. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rolling.html. Accessed: December 20, 2025.
20. Scikit-learn Developers: sklearn.ensemble.RandomForestClassifier — scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html. Accessed: January 8, 2026.
21. Scikit-learn Developers: Probability Calibration. Scikit-learn Documentation. https://scikit-learn.org/stable/modules/calibration.html. Accessed: January 12, 2026.
22. Scikit-learn Developers: sklearn.calibration.CalibratedClassifierCV — scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.calibration.CalibratedClassifierCV.html. Accessed: January 20, 2026.
23. Apache Software Foundation: Apache Parquet. https://parquet.apache.org/. Accessed: February 20, 2026.
24. 24. Electric Power Research Institute (EPRI): Elements of Effective Alarm Management and Display Design. Product ID 1010076 (2005). https://restservice.epri.com/publicdownload/000000000001010076/0/Product.
25. Mossé, D.: Fault Tolerance. Course Materials, University of Pittsburgh. https://people.cs.pitt.edu/~melhem/courses/3410p/ft.pdf. Accessed: February 10, 2026.
26. Scikit-learn Developers: sklearn.metrics.roc_auc_score documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html. Accessed: February 3, 2026.
27. Scikit-learn Developers: sklearn.metrics.f1_score documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html. Accessed: February 10, 2026.
28. Scikit-learn Developers: sklearn.metrics.brier_score_loss documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.brier_score_loss.html. Accessed: January 3, 2026.
29.